

ML Framework for Password Policies

Siddhi Kabadi
Santa Clara University
skabadi@scu.edu

Abstract

As text passwords remain the default for password authentication, there is a growing need for public understanding of password strength, and what features reliably quantify it. Existing researchers rely on the LUDS mechanism—lowercase, uppercase, digits, and symbols, to rank the strength of a password. Prior studies conclude that these quantitative heuristics are ineffective and often overlook linguistic patterns and other common word structures (ex. l33tspeak). To address these disparities, researchers suggest using entropy-based metrics, such as zxcvbn, an open-source library. In this paper, we offer a bridge between both approaches and present a Machine Learning (ML) based framework to evaluate password strength. Specifically, we fine-tuned a BERT model (bert-base-cased) on a labeled dataset of 468,000 passwords to classify entries as either easy, medium, or difficult, and then applied this model on two unlabeled datasets of passwords, one of commonly used passwords, and the other of randomly generated strings. Our model achieved a precision of 87% and an F1 score of 0.81, showing that our framework could efficiently identify patterns and classify passwords. To compare the results of our model to existing password policies, we conducted correlation analysis between the 4 LUDS features and our model's outputs. On the random set, the associated Pearson correlations were the following: 0.85 for length, 0.57 for symbol count, and 0.45 for digit count. Our work also builds on the *zxcvbn: Low-Budget Password Strength Estimation (USENIX Security 2016)* paper, which utilizes predefined patterns such as keyboard proximity and l33tspeak substitutions into its scoring system. *zxcvbn* relies on fixed rules, while our approach uses a BERT model to learn inherent linguistic and structural features when ranking patterns. BERT processes strings bidirectionally, which allows our model to learn contextual dependencies between characters, digits, and symbols in a way that rule-based heuristics fail to. Our results show medium levels of agreement between BERT and *zxcvbn* in password rankings, with differences mostly in passwords ranked in the medium to strong range. This paper offers a methodological contribution to the password security community by using language models for strength evaluation, and brainstorming a novel approach to creating password policies.

CCS Concepts

• Security and privacy → Password systems; • Computing methodologies → Natural language processing.

Keywords

Password Strength, Authentication, BERT, Transformer Models, Machine Learning, Password Policies

1 Introduction

Over the past few years, there has been an unprecedented increase in password breaches, highlighting the fragility and lack of complexity within password creation. Users often create passwords rooted in predictable character structures, such as l33tspeak substitutions, and easy numerical sequences like 12345. There is a general consensus among users to prioritize memorability over security, which then leads to large-scale brute-force and dictionary-based attacks. Researchers have studied different techniques on how to enforce security within password creation, with the most prevalent method being the LUDS framework. LUDS refers to lowercase letters, uppercase letters, digits, and symbols, and password policies tend to hold rules on the numerical count of these characters. However, this framework is weak as users can comply by simply adding a symbol or digit to a weak password. Simply adding one of these characters does not change the overall insecure structure of the password, and instead allows users to create credentials that remain highly guessable. Recent research addresses the shortcomings of LUDS by advocating for entropy-based password policies. The *zxcvbn* library, created by Dropbox, builds onto LUDS by encoding keyboard proximity checks, dictionary substitutions, and other heuristic entropy calculations. *zxcvbn* improves upon the rigid rules of LUDS, but still relies on predefined rules and handcrafted pattern checks. While *zxcvbn* is a better alternative to LUDS, it still fails to study the contextual dependencies within each character of a password, and will not characterize password strength accurately. Attackers have also started gaining access to leaked password datasets and generative AI models, allowing them to use data-driven password cracking strategies. Studies such as *Guess Again (and Again and Again)* explain how attackers can exploit linguistic and structural biases within passwords through mathematically based guesses. Attackers using data-driven models, creates an unparalleled imbalance in cybersecurity. Attackers are using adaptive and data-driven methods to crack passwords, while users are relying on simple handcrafted rules. This imbalance highlights a need for ML based strength estimators in password security. In this research, we study whether contextual language models can accurately determine password strength beyond basic rule-based and entropy-based heuristics. We fine-tune a BERT model on a labeled Kaggle dataset of 468,747 passwords, that span categories of weak, medium, and strong. BERT's bidirectional attention mechanism allows us to study contextual relationships across entire password strings, and learn how characters, digits, and symbols interact to shape a password. This paper makes two contributions. First, we explain how a fine-tuned BERT model holds a strong performance record of 87% accuracy and a F1 score of 0.81 while also studying structural patterns that are overlooked by current password strength estimators. Second, we conduct an graphical analysis of BERT's predictions by using correlation studies and clustering to compare the conclusions of BERT to

LUDS and zxcvbn. We will identify areas where BERT aligns or differs from rule-based approaches, and will contribute to the design of secure password strength by providing data-driven insights.

2 Methodology

Our methodology is designed to evaluate whether language models can classify password strength in a more efficient and holistic manner than traditional rule-based techniques. We fine-tuned a BERT model on a labeled dataset of passwords and evaluated its predictions across two different external datasets. Below, our data sources, model structure, training configuration, and evaluation procedure is outlined.

2.1 Dataset Cleaning and Preparation

Our training dataset was obtained from Kaggle [8], and consisted of two different columns: password and strength. The strength column contains values 0, 1, and 2, which represent easy, medium, and hard, respectively. To prepare and clean the data, we utilized Python’s Pandas library to delete rows with NaN values. We then converted the dataset into a dataframe. Due to computational restraints, we took a sample of 70% of the dataframe, still ensuring that the distribution of the labels remained the same. The model was trained with 70% of the sampled data, and the other 30% was reserved for testing. Our training dataset had a total of 468,747 entries, which are classified as weak, medium, or strong, or 0, 1, or 2, respectively. The dataset consisted of 62,791 weak examples, 347,760 medium examples, and 58,196 strong examples.

2.2 Model Architecture

We implemented the bert-base-cased architecture from HuggingFace’s Transformers library for our model. Bert-base-cased models are case sensitive, which is crucial for our project as distinctions need to be made between lowercase and uppercase letters. We also added a linear classification layer to our token embeddings.

2.3 Tokenization, Encoding, and Training

Passwords were preprocessed using the HuggingFace bert-base-cased tokenizer, which converts individual password strings into numerical input IDs. Each password was standardized to a maximum length of 512 tokens, adding padding to shorter passwords, and truncating the length of longer ones. The final resulting input IDs, attention masks, and strength labels were stored as PyTorch tensors, and were set as TensorDataset objects for batching and loading during the training process. Bert-base-cased was trained for 3 epochs, and the learning rate for training was set to $2e(-5)$ using the AdamW optimizer from the torch.optim library. To ensure that the model was efficiently trained, we tracked the model’s overall loss, the difference between the model’s predictions and the true labels. We extracted pooled outputs, the summary of the sequence, logits, and the model’s predictions by running the outputs through a linear classifier. Cross entropy loss was then used to calculate the total loss by comparing the logits to the true labels. The model updated its parameters based on the loss, and continued iterating. After training for three iterations, we generated the accuracy and F1 score. Using this method, our model achieved an accuracy of approximately 87% and an F1 score of 0.81. These scores indicate

that the trained model was able to correctly identify patterns and classify passwords on the testing dataset.

3 Results

To evaluate the performance of our BERT model, we conducted tests on two different datasets. The first consisted of 10,000 common passwords, and the second consisted of 10,000 randomly generated passwords, where password strings were randomly generated using combinations of letters (uppercase and lowercase), digits, and symbols of arbitrary lengths. For both datasets, we encoded the passwords in the same method used for the password column in the training and testing datasets. Similar to our evaluation step, we passed in these inputs to the model and extracted the outputs and logits. The logits were converted into prediction scores of 0, 1, or 2, and then we created two new data frames with the passwords and their BERT classifications. For the set of randomly generated passwords, the model classified longer strings with high symbol counts as strong. This is supported with a Pearson correlation of 0.85 for length and 0.57 for symbol usage. The commonly used passwords consisted of an inherent bias towards weaker classifications and had correlations of 0.70 for length and 0.45 for digit count. We also applied KMeans clustering to the random data set to further understand patterns in classification. Our results showed that weaker passwords formed homogeneous clusters with a small number of digits and uppercase letters. Strong passwords tended to have more diversity and complexity. Finally, we also compared the conclusions of our BERT model to those of the zxcvbn library. We compared the score rankings of the randomly generated passwords as it did not have an inherent bias of weaker passwords. We found similar agreement levels for passwords ranked as weak. There was a high overlap in the score rankings, except for disagreements in the medium / strong rankings. Our study shows how BERT tends to rank more passwords as medium strength, as it holds a stricter requirement when studying structural and linguistic features of a password. Overall, the mechanism of LUDS and zxcvbn is still relevant in studying password strength, but using a ML based approach allows for a more holistic and thorough check in the structural and linguistic predictability of a password.

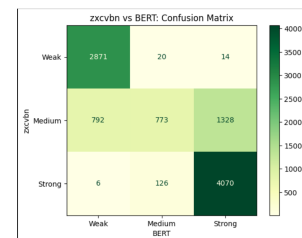


Figure 1: Confusion Matrix: zxcvbn vs BERT

References

- [1] D. L. Wheeler and D. Lowe, “zxcvbn: Low-Budget Password Strength Estimation,” in *Proceedings of the 25th USENIX Security Symposium*, 2016. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_wheeler.pdf

- [2] D. Pasquini, M. Cianfriglia, G. Ateniese, and M. Bernaschi, "Reducing Bias in Modeling Real-world Password Strength via Deep Learning and Dynamic Dictionaries," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021. [Online]. Available: <https://www.usenix.org/system/files/sec21-pasquini.pdf>
- [3] P. G. Kelley, et al., "Guess Again (and Again and Again): Measuring Password Strength by Simulating Password-Cracking Algorithms," *IEEE Transactions on Dependable and Secure Computing*, May 2012. [Online]. Available: <https://ieeexplore.ieee.org/document/6234434>
- [4] Technocrat, "Fine-tuning BERT for Text Classification: A Step-by-Step Guide," Medium, Sep. 13, 2023. [Online]. Available: <https://medium.com/@coderhack.com/fine-tuning-bert-for-text-classification-a-step-by-step-guide-1a1c5f8e8ae1>
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," May 2019. [Online]. Available: <https://arxiv.org/pdf/1810.04805>
- [6] Coverlink, "City of Atlanta Ransomware Case Study," 2018. [Online]. Available: <https://coverlink.com/case-study/city-of-atlanta-ransomware/>
- [7] D. Pasquini, et al., "Presentation: Reducing Bias in Modeling Real-world Password Strength via Deep Learning and Dynamic Dictionaries," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/pasquini>
- [8] Kaggle, "Password Strength Classifier Dataset." [Online]. Available: <https://www.kaggle.com/datasets/bhavikbb/password-strength-classifier-dataset>
- [9] Google, "BERT Base Cased Model," Hugging Face. [Online]. Available: <https://huggingface.co/google-bert/bert-base-cased>
- [10] PyTorch, "Tensor Documentation." [Online]. Available: <https://pytorch.org/docs/stable/tensors.html>
- [11] National Institutes of Health, "A Guide to Understanding Systematic Reviews." [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4291202/>
- [12] Kaggle, "10000 Common Passwords Dataset." [Online]. Available: <https://www.kaggle.com/code/shivamb/10000-common-passwords>
- [13] Kaggle, "Random Passwords Dataset." [Online]. Available: <https://www.kaggle.com/datasets/jeffersonvalandro/password-dataset>
- [14] IBM, "K-means Clustering." [Online]. Available: <https://www.ibm.com/topics/k-means-clustering>
- [15] L. Zhang, J. Stokes, and R. Wang, "Cost-aware Active Learning for Intrusion Detection," arXiv preprint arXiv:1907.11692, 2019. [Online]. Available: <https://arxiv.org/pdf/1907.11692>